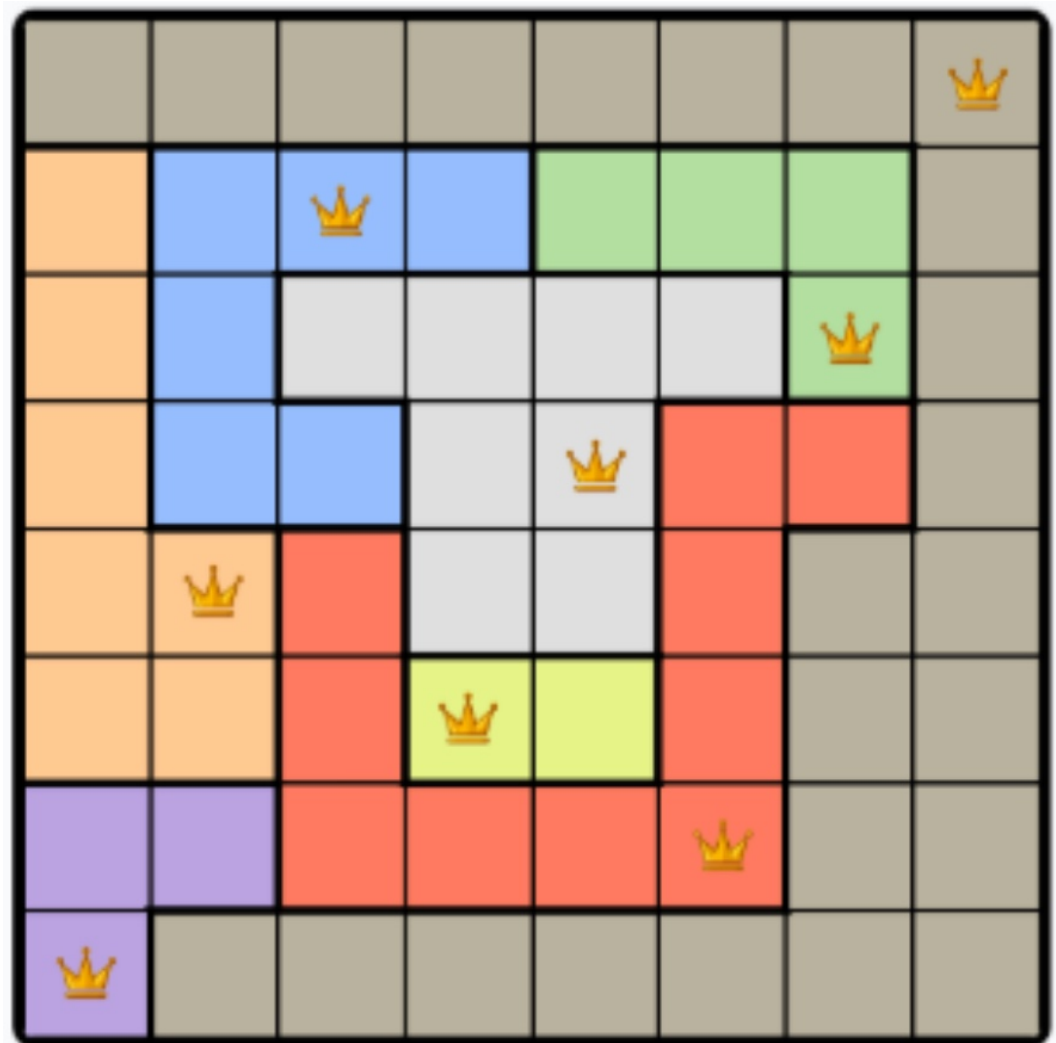


Laporan Tugas Kecil 1

IF2211 STRATEGI ALGORITMA
Penyelesaian Permainan Queens Linked in
Semester II Tahun 2025/2026



Jonathan Kris Wicaksono
13524023

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung



Daftar Isi

Daftar Tabel	ii
Daftar Gambar	iii
1 Implementasi Algoritma	1
1.1 Algoritma Brute Force	1
1.2 Fitur Tambahan Program	2
2 Source Code	3
3 Pengujian	12
3.1 Test 1: Papan berukuran 9×9 (in1.txt)	12
3.2 Test 2: Papan berukuran 11×11 (in2.txt)	13
3.3 Test 3: Papan berukuran 1×1 (in3.txt)	14
3.4 Test 4: Papan berukuran 9×9 — Tidak Ada Solusi (in4.txt)	15
3.5 Test 5: Input Tidak Valid — Panjang Baris Berbeda (in5.txt)	16
4 Lampiran	18



Daftar Tabel

3.1	Input Test 1	12
3.2	Input Test 2	13
3.3	Input Test 3	14
3.4	Input Test 4	15
3.5	Input Test 5	16
4.1	Evaluasi	18



Daftar Gambar

3.1.1 Output GUI — Test 1	13
3.2.1 Output GUI — Test 2	14
3.3.1 Output GUI — Test 3	15
3.4.1 Output GUI — Test 4 (Tidak Ada Solusi)	16
3.5.1 Output GUI — Test 5 (Input Tidak Valid)	17

1 Implementasi Algoritma

Dalam permainan Queens LinkedIn, tujuannya adalah menempatkan satu *queen* pada setiap baris di papan berukuran $N \times M$ yang terbagi menjadi daerah warna, dengan syarat: tepat satu queen pada setiap baris, kolom, dan daerah warna, serta tidak boleh ada dua queen yang saling bersebelahan (termasuk diagonal). Program ini ditulis dalam bahasa C++ dengan GUI menggunakan library SFML 3.0 dan mengimplementasikan algoritma Brute Force.

1.1 Algoritma Brute Force

Algoritma *brute force* yang diimplementasikan bersifat murni, yaitu mengenumerasi seluruh kemungkinan penempatan queen tanpa heuristik apapun.

Berikut adalah langkah-langkah detail dari algoritma brute force yang diimplementasikan pada fungsi `solve_bf()`:

1. Program menerima masukan berupa *grid* berukuran $N \times M$ yang dibaca dari file `.txt`. Setiap huruf pada *grid* merepresentasikan daerah warna.
2. Dibuat sebuah *array* `curr` berukuran N , diinisialisasi dengan $[0, 0, \dots, 0]$. Elemen `curr[i]` menyatakan kolom tempat queen diletakkan pada baris ke- i .
3. Untuk setiap konfigurasi `curr`, dilakukan validasi terhadap seluruh syarat permainan:
 - **Syarat kolom:** Tidak boleh ada dua queen pada kolom yang sama, diperiksa dengan membandingkan `curr[i]` terhadap `curr[prev]` untuk seluruh baris sebelumnya.
 - **Syarat warna:** Tidak boleh ada dua queen pada daerah warna yang sama, diperiksa dengan membandingkan `grid[i][curr[i]]` terhadap `grid[prev][curr[prev]]`.
 - **Syarat adjacency:** Queen pada baris berurutan tidak boleh berada pada kolom yang bersebelahan, diperiksa dengan kondisi $|curr[i] - curr[i-1]| \leq 1$.
4. Jika seluruh syarat terpenuhi, konfigurasi disimpan sebagai solusi.
5. Jika tidak valid, konfigurasi di-*increment*: `curr[N-1]` dinaikkan 1; jika melebihi M , di-*reset* ke 0 dan `curr[N-2]` dinaikkan, dan seterusnya.
6. Proses berulang hingga solusi ditemukan atau seluruh M^N konfigurasi habis diperiksa.

Algoritma ini memiliki kompleksitas waktu $O(M^N \cdot N)$ karena setiap konfigurasi memerlukan $O(N)$ untuk validasi.

Algorithm 1 Algoritma Brute Force Queens

```
1: procedure SOLVEBRUTEFORCE(grid, N, M)
2:   curr  $\leftarrow$  [0, 0, ..., 0] ▷ Array berukuran N
3:   while true do
4:     iter  $\leftarrow$  iter + 1
5:     valid  $\leftarrow$  true
6:     for i  $\leftarrow$  0 to N - 1 do
7:       c  $\leftarrow$  curr[i]
8:       if c  $\geq$  M then valid  $\leftarrow$  false; break
9:       end if
10:      for prev  $\leftarrow$  0 to i - 1 do
11:        if curr[prev] = c then valid  $\leftarrow$  false; break
12:        end if ▷ Kolom sama
13:        if grid[prev][curr[prev]] = grid[i][c] then valid  $\leftarrow$  false; break
14:        end if ▷ Warna sama
15:      end for
16:      if i > 0 and |c - curr[i - 1]|  $\leq$  1 then valid  $\leftarrow$  false; break
17:      end if ▷ Adjacency
18:    end for
19:    if valid then return curr ▷ Solusi ditemukan
20:    end if
21:    ▷ Increment ke konfigurasi berikutnya
22:    idx  $\leftarrow$  N - 1
23:    while idx  $\geq$  0 do
24:      curr[idx]  $\leftarrow$  curr[idx] + 1
25:      if curr[idx] < M then break
26:      end if
27:      curr[idx]  $\leftarrow$  0; idx  $\leftarrow$  idx - 1
28:    end while
29:    if idx < 0 then return None ▷ Tidak ada solusi
30:    end if
31:  end while
32: end procedure
```

1.2 Fitur Tambahan Program

Selain algoritma pencarian, program juga mengimplementasikan:

1. **GUI (SFML 3.0):** Panel kontrol dengan tombol-tombol interaktif (BROWSE, LOAD, SOLVE, STOP, SAVE TXT, SAVE IMG), pemilihan algoritma, dan slider kecepatan visualisasi.
2. **Live Visualization:** Setiap iterasi pencarian divisualisasikan secara *real-time* pada papan GUI. Kecepatan dapat diatur melalui slider (TURBO / FAST / SLOW).
3. **Validasi Input:** Program memverifikasi:
 - Seluruh baris memiliki panjang yang sama
 - Karakter hanya boleh berupa huruf A-Z
 - Jumlah region warna unik harus sama dengan jumlah baris (*N*)
4. **Output as Text:** Solusi dapat disimpan sebagai file `.txt` melalui tombol SAVE TXT, lengkap dengan waktu pencarian dan jumlah iterasi.
5. **Output as Image:** Solusi dapat disimpan sebagai file PNG melalui tombol SAVE IMG.

2 Source Code

Berikut adalah source code program penyelesaian permainan Queens LinkedIn yang ditulis dalam bahasa C++ dengan GUI menggunakan SFML 3.0. Kode terdapat dalam satu file `src/main.cpp`.

```
1 #include <SFML/Graphics.hpp>
2 #include <algorithm>
3 #include <chrono>
4 #include <cmath>
5 #include <cstdio>
6 #include <filesystem>
7 #include <fstream>
8 #include <iostream>
9 #include <memory>
10 #include <optional>
11 #include <string>
12 #include <thread>
13 #include <vector>
14 #ifdef _WIN32
15 #define PCLOSE_CMD _pclose
16 #define POPEN_FUNC _popen
17 #else
18 #define PCLOSE_CMD pclose
19 #define POPEN_FUNC popen
20 #endif
21 using namespace std;
22 using namespace sf;
23
24 const unsigned int W_WIDTH = 1100;
25 const unsigned int W_HEIGHT = 750;
26 const Color C_BG(30, 30, 30);
27 const Color C_PANEL(44, 62, 80);
28 const Color C_BTN(52, 73, 94);
29 const Color C_BTN_H(93, 109, 126);
30 const Color C_BTN_A(46, 204, 113);
31 const Color C_BTN_R(231, 76, 60);
32 const Color C_BTN_B(52, 152, 219);
33 const Color C_TXT(236, 240, 241);
34 const Color C_SUB(189, 195, 199);
35
36 int n = 0, m = 0;
37 vector<string> grid;
38 vector<int> ans;
39 int64_t iter = 0;
40 bool found = false, solving = false, stop = false;
41 int delay_ms = 0, mode = 2;
42 double elapsed_time = 0.0;
43 string file_path = "";
44 string save_msg = "";
45 Texture q_tex;
46 bool has_tex = false;
47
48 string get_file_dialog(int mode) {
49     string cmd;
50     string res = "";
51     char buf[128];
52     #ifdef _WIN32
53     cmd = "C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe -command \"Add-Type -AssemblyName \"
54         \"System.Windows.Forms; $f = New-Object System.Windows.Forms.\";
55     cmd += (mode == 0 ? \"OpenFileDialog\" : \"SaveFileDialog\");
56     cmd += \"; $f.Filter = '\";
57     if (mode == 0)
58         cmd += \"Text Files (*.txt)|*.txt|All Files (*.*)|*.*\";
59     else if (mode == 1)
60         cmd += \"PNG Image (*.png)|*.png\";
61     else
62         cmd += \"Text Files (*.txt)|*.txt\";
63     cmd += \"'; $f.ShowDialog() | Out-Null; $f.FileName\";
64     #elif __APPLE__
65     if (mode == 0) {
66         cmd = \"osascript -e 'POSIX path of (choose file of type {\\\"txt\\\"} with prompt \"
67         \"Select Board\\')'\";
68     } else if (mode == 1) {
```

```
68     cmd = "osascript -e 'POSIX path of (choose file name with prompt \"Save Solution\"
        default name \"
        \"solution.png\")'";
69
70 } else {
71     cmd = "osascript -e 'POSIX path of (choose file name with prompt \"Save Solution\"
        default name \"
        \"solution.txt\")'";
72
73 }
74 #else
75 cmd = "zenity --file-selection";
76 if (mode == 0)
77     cmd += " --file-filter=*.txt";
78 else if (mode == 1)
79     cmd += " --save --filename=solution.png --confirm-overwrite";
80 else
81     cmd += " --save --filename=solution.txt --confirm-overwrite";
82 #endif
83 FILE *pipe = POPEN_FUNC(cmd.c_str(), "r");
84 if (!pipe) return "";
85 while (fgets(buf, sizeof buf, pipe) != NULL) {
86     res += buf;
87 }
88 PCLOSE_CMD(pipe);
89 res.erase(remove(res.begin(), res.end(), '\n'), res.end());
90 res.erase(remove(res.begin(), res.end(), '\r'), res.end());
91 return res;
92 }
93 Color get_col(char c) {
94     if (c == ' ' || c == 0) return Color(20, 20, 20);
95     int v = (c >= 'a') ? (c - 'a') : (c - 'A');
96     return Color((v * 60 + 20) % 255, (v * 90 + 40) % 255, (v * 40 + 100) % 255);
97 }
98 void sync_vis(const vector<int> &curr) {
99     if (delay_ms > 0) {
100         ans = curr;
101         this_thread::sleep_for(chrono::microseconds(delay_ms));
102     }
103 }
104 string validate_board() {
105     if (n == 0) return "Board is empty";
106     for (int i = 0; i < n; i++)
107         for (int j = 0; j < m; j++) {
108             char c = grid[i][j];
109             if (c < 'A' || c > 'Z') return "Invalid char '" + string(1, c) + "'\nat row " +
                to_string(i + 1);
110         }
111     vector<bool> seen(26, false);
112     int cnt = 0;
113     for (int i = 0; i < n; i++)
114         for (int j = 0; j < m; j++) {
115             int idx = grid[i][j] - 'A';
116             if (!seen[idx]) {
117                 seen[idx] = true;
118                 cnt++;
119             }
120         }
121     if (cnt != n) return "Need " + to_string(n) + " regions,\nfound " + to_string(cnt);
122     return "";
123 }
124 void solve_bf() {
125     vector<int> curr(n, 0);
126     while (!stop) {
127         iter++;
128         sync_vis(curr);
129         if (delay_ms == 0 && iter % 5000 == 0) ans = curr;
130         bool ok = true;
131         for (int i = 0; i < n; i++) {
132             int c = curr[i];
133             if (c >= m) {
134                 ok = false;
135                 break;
136             }
137             char col = grid[i][c];
138             for (int prev = 0; prev < i; prev++) {
139                 int pc = curr[prev];
140                 if (pc == c) {
```



```
141         ok = false;
142         break;
143     }
144     if (grid[prev][pc] == col) {
145         ok = false;
146         break;
147     }
148 }
149 if (!ok) break;
150 if (i > 0) {
151     if (abs(c - curr[i - 1]) <= 1) {
152         ok = false;
153         break;
154     }
155 }
156 }
157 if (ok) {
158     ans = curr;
159     found = true;
160     return;
161 }
162 int idx = n - 1;
163 while (idx >= 0) {
164     curr[idx]++;
165     if (curr[idx] < m) break;
166     curr[idx] = 0;
167     idx--;
168 }
169 if (idx < 0) return;
170 }
171 }
172 bool safe(int r, int c, const vector<int> &curr) {
173     if (c >= m) return false;
174     char col = grid[r][c];
175     for (int pr = 0; pr < r; pr++) {
176         int pc = curr[pr];
177         if (pc == c) return false;
178         if (grid[pr][pc] == col) return false;
179         if (pr == r - 1 && abs(pc - c) <= 1) return false;
180     }
181     return true;
182 }
183 void solve_bt(int r, vector<int> &curr) {
184     if (found || stop) return;
185     if (r == n) {
186         ans = curr;
187         found = true;
188         return;
189     }
190     for (int c = 0; c < m; c++) {
191         iter++;
192         curr[r] = c;
193         sync_vis(curr);
194         if (delay_ms == 0 && iter % 2000 == 0) ans = curr;
195         if (safe(r, c, curr)) {
196             solve_bt(r + 1, curr);
197             if (found) return;
198         }
199         curr[r] = -1;
200     }
201 }
202 void run() {
203     auto start = chrono::high_resolution_clock::now();
204     vector<int> tmp(n, (mode == 1 ? 0 : -1));
205     ans = tmp;
206     if (mode == 1)
207         solve_bf();
208     else
209         solve_bt(0, tmp);
210     auto end = chrono::high_resolution_clock::now();
211     elapsed_time = chrono::duration<double, milli>(end - start).count();
212     solving = false;
213 }
214 void draw(RenderTarget &t, float w, float h, float sx, float sy) {
215     if (n == 0 || m == 0) return;
216     float ratio_g = (float)m / n;
```

```
217 float ratio_a = w / h;
218 float sz, fw, fh;
219 if (ratio_g > ratio_a) {
220     fw = w;
221     sz = w / m;
222     fh = sz * n;
223 } else {
224     fh = h;
225     sz = h / n;
226     fw = sz * m;
227 }
228 float ox = sx + (w - fw) / 2.0f;
229 float oy = sy + (h - fh) / 2.0f;
230 for (int i = 0; i < n; i++) {
231     for (int j = 0; j < m; j++) {
232         RectangleShape cell(Vector2f(sz - 2.f, sz - 2.f));
233         cell.setPosition({ox + j * sz + 1.f, oy + i * sz + 1.f});
234         cell.setFill(Color(get_col(grid[i][j]));
235         t.draw(cell);
236         if (i < (int)ans.size() && ans[i] == j) {
237             if (has_tex) {
238                 Sprite s(q_tex);
239                 Vector2u ts = q_tex.getSize();
240                 float sc = (sz * 0.8f) / max(ts.x, ts.y);
241                 s.setScale({sc, sc});
242                 s.setOrigin({ts.x / 2.f, ts.y / 2.f});
243                 s.setPosition({ox + j * sz + sz / 2.f, oy + i * sz + sz / 2.f});
244                 t.draw(s);
245             } else {
246                 CircleShape c(sz * 0.35f);
247                 c.setFill(Color::Black);
248                 c.setOutlineThickness(2.f);
249                 c.setOutlineColor(Color::White);
250                 c.setOrigin({sz * 0.35f, sz * 0.35f});
251                 c.setPosition({ox + j * sz + sz / 2.f, oy + i * sz + sz / 2.f});
252                 t.draw(c);
253             }
254         }
255     }
256 }
257 }
258 void save_txt() {
259     if (!found || n == 0) return;
260     string path = get_file_dialog(2);
261     if (path.empty() || path == "ERROR") return;
262     if (path.length() < 4 || path.substr(path.length() - 4) != ".txt") {
263         path += ".txt";
264     }
265     ofstream out(path);
266     if (!out) {
267         save_msg = "Error saving file!";
268         return;
269     }
270     for (int i = 0; i < n; i++) {
271         for (int j = 0; j < m; j++) {
272             if (ans[i] == j)
273                 out << '#';
274             else
275                 out << grid[i][j];
276         }
277         out << '\n';
278     }
279     out << "\nWaktu pencarian: " << (int64_t)elapsed_time << " ms\n";
280     out << "Banyak kasus yang ditinjau: " << iter << " kasus\n";
281     out.close();
282     save_msg = "Saved: " + path;
283     cout << save_msg << endl;
284 }
285 void save_img() {
286     if (!found || n == 0) return;
287     string path = get_file_dialog(1);
288     if (path.empty() || path == "ERROR") return;
289     unsigned int iw = 1200;
290     unsigned int ih = (unsigned int)(1200.0f * ((float)n / m));
291     if (ih < 600) ih = 600;
292     RenderTexture rt;
```



```
293     if (!rt.resize({iw, ih})) return;
294     rt.clear(C_BG);
295     draw(rt, (float)iw, (float)ih, 0.f, 0.f);
296     rt.display();
297     if (rt.getTexture().copyToImage().saveToFile(path)) {
298         save_msg = "Saved: " + path;
299         cout << save_msg << endl;
300     } else {
301         save_msg = "Error saving image!";
302     }
303 }
304 struct Btn {
305     RectangleShape s;
306     Text t;
307     bool h = false, a = false;
308     Btn(Vector2f sz, Vector2f p, string l, Font &f, Color c = C_BTN) : t(f) {
309         s.setSize(sz);
310         s.setPosition(p);
311         s.setFillColor(c);
312         t.setString(l);
313         t.setCharacterSize(14);
314         t.setFillColor(C_TXT);
315         FloatRect b = t.getLocalBounds();
316         t.setPosition({floor(p.x + (sz.x - b.size.x) / 2.f), floor(p.y + (sz.y - b.size.y) /
317             2.f - 4.f)});
318     }
319     bool upd(Vector2f mp, bool clk) {
320         h = s.getGlobalBounds().contains(mp);
321         if (a)
322             s.setFillColor(C_BTN_A);
323         else if (h)
324             s.setFillColor(C_BTN_H);
325         else if (s.getFillColor() == C_BTN_H || s.getFillColor() == C_BTN_A) {
326             if (!a) s.setFillColor(C_BTN);
327         }
328         return h && clk;
329     }
330     void drw(RenderWindow &w) {
331         w.draw(s);
332         w.draw(t);
333     }
334 };
335 struct Inp {
336     RectangleShape s;
337     Text t;
338     string val;
339     Inp(Vector2f sz, Vector2f p, Font &f) : t(f) {
340         s.setSize(sz);
341         s.setPosition(p);
342         s.setFillColor(Color(50, 50, 50));
343         s.setOutlineColor(Color::White);
344         s.setOutlineThickness(1.f);
345         t.setCharacterSize(14);
346         t.setFillColor(Color::Yellow);
347         t.setPosition({p.x + 5.f, p.y + 7.f});
348         t.setString("Select file...");
349     }
350     void set(string v) {
351         val = v;
352         size_t sl = v.find_last_of("/\\");
353         string d = (sl != string::npos) ? v.substr(sl + 1) : v;
354         if (d.length() > 25) d = "..." + d.substr(d.length() - 22);
355         t.setString(d);
356         t.setFillColor(Color::White);
357     }
358     void drw(RenderWindow &w) {
359         w.draw(s);
360         w.draw(t);
361     }
362 };
363 struct Sld {
364     RectangleShape tr, hd;
365     Text lbl;
366     float mn, mx, cur;
367     bool drag = false;
368     Sld(Vector2f p, float w, float minv, float maxv, Font &f) : lbl(f) {
```

```
368     mn = minv;
369     mx = maxv;
370     cur = minv;
371     tr.setSize({w, 6.f});
372     tr.setPosition(p);
373     tr.setFillColor(Color(149, 165, 166));
374     hd.setSize({16.f, 20.f});
375     hd.setOrigin({8.f, 10.f});
376     hd.setFillColor(Color::White);
377     recalc();
378     lbl.setCharacterSize(13);
379     lbl.setFillColor(C_TXT);
380     lbl.setPosition({p.x, p.y - 22.f});
381     lbl.setString("Speed: TURBO");
382 }
383 void recalc() {
384     float pct = (cur - mn) / (mx - mn);
385     hd.setPosition({tr.getPosition().x + pct * tr.getSize().x, tr.getPosition().y + tr.
        getSize().y / 2.f});
386 }
387 void upd(RenderWindow &w) {
388     Vector2f mp = w.mapPixelToCoords(Mouse::getPosition(w));
389     if (Mouse::isButtonPressed(Mouse::Button::Left)) {
390         FloatRect hb = tr.getGlobalBounds();
391         hb.position.y -= 10.f;
392         hb.size.y += 20.f;
393         if (hd.getGlobalBounds().contains(mp) || hb.contains(mp)) drag = true;
394     } else
395         drag = false;
396     if (drag) {
397         float mx_pos = max(tr.getPosition().x, min(mp.x, tr.getPosition().x + tr.getSize().
            x));
398         float pct = (mx_pos - tr.getPosition().x) / tr.getSize().x;
399         cur = mn + pct * (mx - mn);
400         recalc();
401         if (cur <= 50) {
402             lbl.setString("Speed: TURBO");
403             lbl.setFillColor(Color::Cyan);
404         } else if (cur < 5000) {
405             lbl.setString("Speed: FAST");
406             lbl.setFillColor(Color::Green);
407         } else {
408             lbl.setString("Speed: SLOW");
409             lbl.setFillColor(Color(255, 100, 100));
410         }
411     }
412 }
413 void drw(RenderWindow &w) {
414     w.draw(tr);
415     w.draw(hd);
416     w.draw(lbl);
417 }
418 };
419 int main() {
420     RenderWindow w(VideoMode(Vector2u(W_WIDTH, W_HEIGHT)), "Queens Solver");
421     w.setFramerateLimit(60);
422     Font f;
423     if (!f.openFromFile("C:/Windows/Fonts/arial.ttf") &&
424         !f.openFromFile("/System/Library/Fonts/Supplemental/Arial.ttf") && !f.openFromFile(
        "font.ttf")) {
425     }
426     vector<string> p_tex = {"queen.png", "src/queen.png"};
427     for (auto &p : p_tex)
428         if (q_tex.loadFromFile(p)) {
429             has_tex = true;
430             q_tex.setSmooth(true);
431             break;
432         }
433     RectangleShape pn(Vector2f(300.f, (float)W_HEIGHT));
434     pn.setFillColor(C_PANEL);
435     Text tit(f, "QUEENS SOLVER", 24);
436     tit.setPosition({20.f, 20.f});
437     tit.setStyle(Text::Bold);
438     Text nm(f, "Made by Jonathan Kris Wicaksono - 13524023", 12);
439     nm.setPosition({20.f, 50.f});
440     nm.setFillColor(C_SUB);
```

```
441 Text l_f(f, "File:", 14);
442 l_f.setPosition({20.f, 90.f});
443 Inp inp({260.f, 30.f}, {20.f, 115.f}, f);
444 Btn b_brs({260.f, 30.f}, {20.f, 155.f}, "BROWSE", f, C_BTN_B);
445 Btn b_ld({260.f, 35.f}, {20.f, 195.f}, "LOAD", f);
446 Text l_m(f, "Algo:", 14);
447 l_m.setPosition({20.f, 250.f});
448 Btn b_m1({260.f, 30.f}, {20.f, 275.f}, "Brute Force", f);
449 Btn b_m2({260.f, 30.f}, {20.f, 315.f}, "Backtrack", f);
450 b_m2.a = true;
451 Sld sld({20.f, 390.f}, 260.f, 0.f, 300000.f, f);
452 Btn b_run({260.f, 50.f}, {20.f, 440.f}, "SOLVE", f, C_BTN_A);
453 Btn b_stp({260.f, 40.f}, {20.f, 500.f}, "STOP", f, C_BTN_R);
454 Btn b_svt({260.f, 35.f}, {20.f, 620.f}, "SAVE TXT", f, C_BTN_A);
455 Btn b_sv({260.f, 35.f}, {20.f, 660.f}, "SAVE IMG", f, C_BTN_B);
456 Text st(f, "Idle", 14);
457 st.setPosition({20.f, 560.f});
458 st.setLineSpacing(1.2f);
459 while (w.isOpen()) {
460     while (const optional ev = w.pollEvent()) {
461         if (ev->is<Event::Closed>()) {
462             stop = true;
463             if (solving) this_thread::sleep_for(chrono::milliseconds(200));
464             w.close();
465         }
466         if (const auto *mb = ev->getIf<Event::MouseButtonPressed>()) {
467             if (mb->button == Mouse::Button::Left) {
468                 Vector2f mp = w.mapPixelToCoords(Mouse::getPosition(w));
469                 if (!solving) {
470                     if (b_brs.upd(mp, true)) {
471                         string p = get_file_dialog(0);
472                         if (!p.empty() && p != "ERROR") {
473                             file_path = p;
474                             inp.set(p);
475                         }
476                     }
477                     if (b_ld.upd(mp, true)) {
478                         ifstream in(file_path);
479                         if (in) {
480                             string l;
481                             grid.clear();
482                             while (getline(in, l)) {
483                                 l.erase(remove(l.begin(), l.end(), '\r'), l.end());
484                                 l.erase(remove(l.begin(), l.end(), ' '), l.end());
485                                 l.erase(remove(l.begin(), l.end(), '\t'), l.end());
486                                 if (l.empty()) continue;
487                                 grid.push_back(l);
488                             }
489                             n = grid.size();
490                             bool valid = (n > 0);
491                             if (valid) {
492                                 m = grid[0].length();
493                                 for (int i = 1; i < n; i++) {
494                                     if ((int)grid[i].length() != m) {
495                                         valid = false;
496                                         break;
497                                     }
498                                 }
499                             }
500                             if (valid && m > 0) {
501                                 string err = validate_board();
502                                 if (err.empty()) {
503                                     ans.assign(n, -1);
504                                     found = false;
505                                     iter = 0;
506                                     save_msg = "";
507                                     st.setString("Loaded " + to_string(n) + "x" + to_string(m));
508                                     st.setFill(Color::White);
509                                 } else {
510                                     st.setString("Invalid:\n" + err);
511                                     st.setFill(Color::Red);
512                                     grid.clear();
513                                     n = 0;
514                                     m = 0;
515                                 }
516                             } else {
```

```
517         st.setString("Invalid board:\nRows must have "  
518                        "equal length");  
519         st.setFillColor(Color::Red);  
520         grid.clear();  
521         n = 0;  
522         m = 0;  
523     }  
524     } else {  
525         st.setString("File error");  
526         st.setFillColor(Color::Red);  
527     }  
528 }  
529 if (b_m1.upd(mp, true)) {  
530     mode = 1;  
531     b_m1.a = true;  
532     b_m2.a = false;  
533 }  
534 if (b_m2.upd(mp, true)) {  
535     mode = 2;  
536     b_m1.a = false;  
537     b_m2.a = true;  
538 }  
539 if (n > 0 && b_run.upd(mp, true)) {  
540     found = false;  
541     iter = 0;  
542     stop = false;  
543     solving = true;  
544     save_msg = "";  
545     thread(run).detach();  
546 }  
547 if (found && b_svt.upd(mp, true)) {  
548     save_txt();  
549 }  
550 if (found && b_sv.upd(mp, true)) {  
551     save_img();  
552 }  
553 }  
554 if (solving && b_stp.upd(mp, true)) stop = true;  
555 }  
556 }  
557 }  
558 Vector2f mp = w.mapPixelToCoords(Mouse::getPosition(w));  
559 sld.upd(w);  
560 delay_ms = (int)sld.cur;  
561 if (!solving) {  
562     b_brs.upd(mp, false);  
563     b_ld.upd(mp, false);  
564     b_m1.upd(mp, false);  
565     b_m2.upd(mp, false);  
566     b_run.upd(mp, false);  
567     b_stp.upd(mp, false);  
568     if (found) b_svt.upd(mp, false);  
569     if (found) b_sv.upd(mp, false);  
570 }  
571 if (solving) {  
572     st.setString("Iter: " + to_string(iter));  
573     st.setFillColor(Color::Yellow);  
574 } else if (save_msg != "") {  
575     st.setString(save_msg);  
576     st.setFillColor(Color::Cyan);  
577     if (save_msg.length() > 35) st.setCharacterSize(10);  
578 } else if (found) {  
579     st.setString("SOLVED\nIter: " + to_string(iter) + "\nTime: " + to_string((int64_t)  
elapsed_time) + "ms");  
580     st.setFillColor(Color::Green);  
581     st.setCharacterSize(14);  
582 } else if (stop) {  
583     st.setString("STOPPED\nIter: " + to_string(iter) + "\nTime: " + to_string((int64_t)  
elapsed_time) + "ms");  
584     st.setFillColor(Color(255, 100, 100));  
585 } else if (n > 0 && iter > 0) {  
586     st.setString("FAILED\nIter: " + to_string(iter) + "\nTime: " + to_string((int64_t)  
elapsed_time) + "ms");  
587     st.setFillColor(Color::Red);  
588 }  
589 w.clear(C_BG);
```



```
590     w.draw(pn);
591     w.draw(tit);
592     w.draw(nm);
593     w.draw(l_f);
594     inp.drw(w);
595     b_brs.drw(w);
596     b_ld.drw(w);
597     w.draw(l_m);
598     b_m1.drw(w);
599     b_m2.drw(w);
600     sld.drw(w);
601     b_run.drw(w);
602     b_stp.drw(w);
603     w.draw(st);
604     if (found && !solving) b_svt.drw(w);
605     if (found && !solving) b_sv.drw(w);
606     draw(w, W_WIDTH - 340.f, W_HEIGHT - 40.f, 320.f, 20.f);
607     w.display();
608 }
609 return 0;
610 }
```

Listing 2.1: main.cpp — Aplikasi Penyelesaian Queens LinkedIn (C++/SFML)

3 Pengujian

Program yang dibuat dapat menyelesaikan papan Queens LinkedIn dengan masukan berupa file `.txt` melalui antarmuka GUI berbasis SFML. Berikut adalah hasil pengujian menggunakan 5 test case yang berbeda, beserta penanganan input tidak valid.

3.1 Test 1: Papan berukuran 9×9 (in1.txt)

Tabel 3.1: Input Test 1

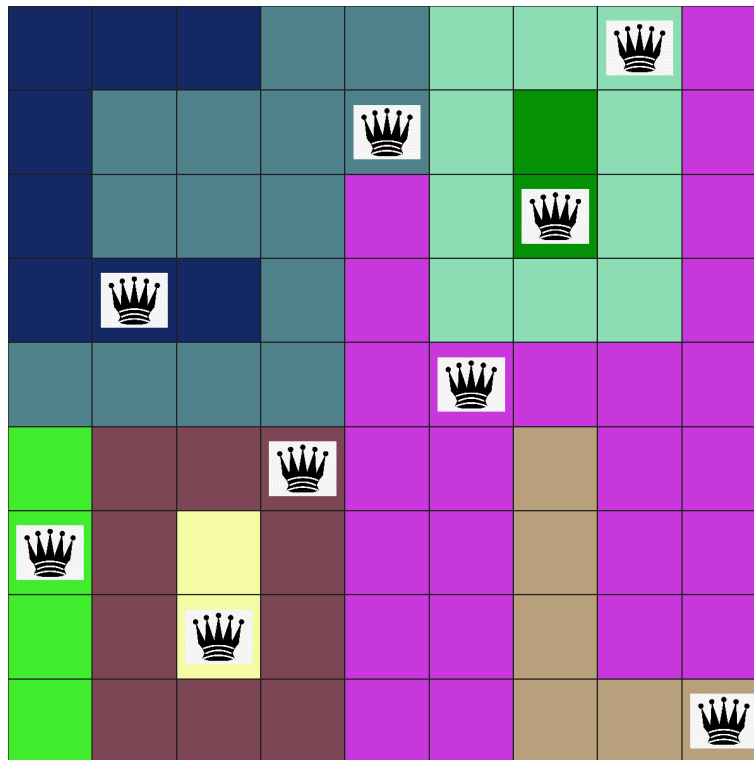
AAABBCCCD
ABBBBCECD
ABBBDCEDD
AAABDCCCD
BBBBDDDDD
FGGGDDHDD
FGIGDDHDD
FGIGDDHDD
FGGGDDHHH

Program menyediakan dua format output: file teks (`.txt`) dan gambar (`.png`). Output teks berisi papan dengan posisi queen ditandai #, waktu pencarian, dan jumlah iterasi. Output gambar menampilkan visualisasi GUI.

Output sebagai File TXT

```
1 AAABBCC#D
2 ABBB#CECD
3 ABBBDC#CD
4 A#ABDCCCD
5 BBBBD#DDD
6 FGG#DDHDD
7 #GIGDDHDD
8 FG#GDDHDD
9 FGGGDDHH#
10
11 Waktu pencarian: 1461 ms
12 Banyak kasus yang ditinjau: 323741637 kasus
```


Output sebagai Gambar (PNG)



Gambar 3.1.1: Output GUI — Test 1

3.2 Test 2: Papan berukuran 11×11 (in2.txt)

Test case ini menggunakan papan berukuran 11×11 dimana setiap baris merupakan satu region warna.

Tabel 3.2: Input Test 2

```

AAAAAAAAAA
BBBBBBBBBB
CCCCCCCCCC
DDDDDDDDDD
EEEEEEEEEE
FFFFFFFFFF
GGGGGGGGGG
HHHHHHHHHH
IIIIIIIII
JJJJJJJJJJ
KKKKKKKKKK

```

Setiap baris terdiri dari satu huruf yang sama (11 kolom), sehingga setiap baris adalah satu region warna.

Program menyediakan dua format output: file teks (.txt) dan gambar (.png).

Output sebagai File TXT

```

1 #AAAAAAAAAA
2 BB#BBBBBBBB
3 CCCC#CCCCC

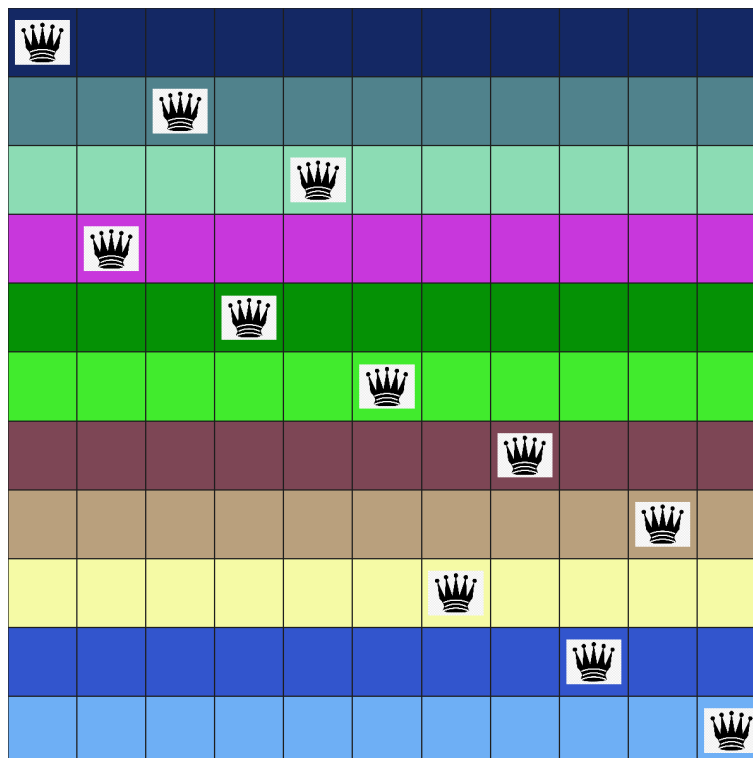
```

```

4 D#DDDDDDDD
5 EEE#EEEEEEE
6 FFFF#FFFFF
7 GGGGGG#GGG
8 HHHHHHHH#H
9 IIIIII#IIII
10 JJJJJJJJ#JJ
11 KKKKKKKKK#
12
13 Waktu pencarian: 17135 ms
14 Banyak kasus yang ditinjau: 5599053306 kasus

```

Output sebagai Gambar (PNG)



Gambar 3.2.1: Output GUI — Test 2

3.3 Test 3: Papan berukuran 1×1 (in3.txt)

Tabel 3.3: Input Test 3

A

Papan berukuran 1×1 dengan satu region. Queen langsung ditempatkan pada satu-satunya sel. Program menyediakan dua format output: file teks (.txt) dan gambar (.png).

Output sebagai File TXT

```

1 #
2
3 Waktu pencarian: 0 ms
4 Banyak kasus yang ditinjau: 1 kasus

```

Output sebagai Gambar (PNG)



Gambar 3.3.1: Output GUI — Test 3

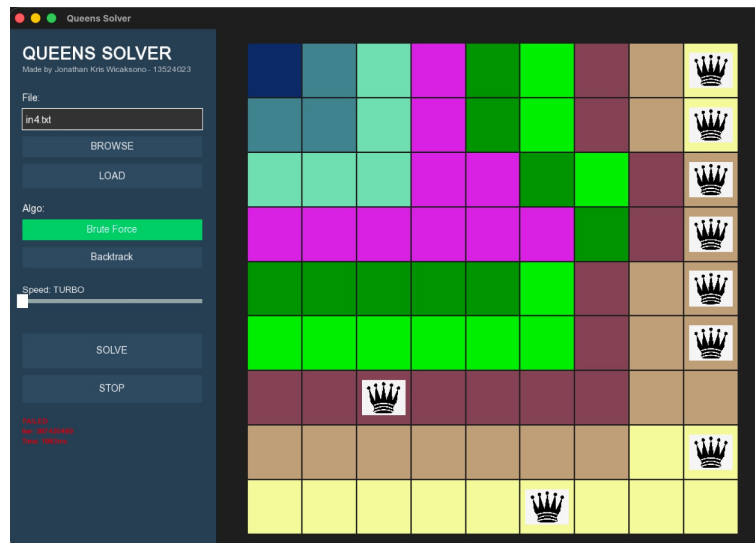
3.4 Test 4: Papan berukuran 9×9 — Tidak Ada Solusi (in4.txt)

Tabel 3.4: Input Test 4

ABCDEFGHI
BBCDEFGHI
CCCDDEFGH
DDDDDEGH
EEEEFGHH
FFFFFGHH
GGGGGGHH
HHHHHHII
IIIIIII

Karena papan ini tidak memiliki solusi yang valid (region tidak memungkinkan penempatan queen yang memenuhi semua aturan), program tidak menghasilkan file solusi .txt.

Output sebagai Gambar (PNG)



Gambar 3.4.1: Output GUI — Test 4 (Tidak Ada Solusi)

3.5 Test 5: Input Tidak Valid — Panjang Baris Berbeda (in5.txt)

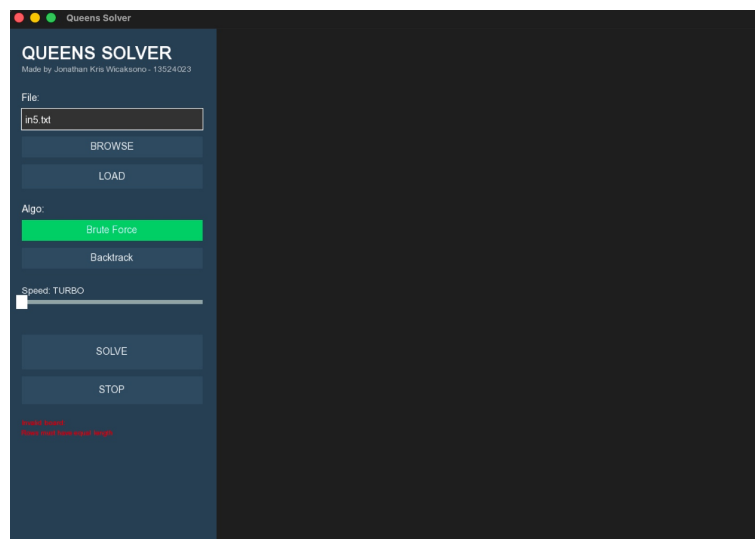
Tabel 3.5: Input Test 5

A	B	C	D	E	F	G	H	I	J	J
B	B	C	D	E	F	G	H	I		
C	C	C	D	D	D	E	F	G	H	
D	D	D	D	D	D	D	E	G	H	
E	E	E	E	E	E	F	G	H	H	
F	F	F	F	F	F	F	G	H	H	
G	G	G	G	G	G	G	G	G	H	H
H	H	H	H	H	H	H	H	H	H	I
I	I	I	I	I	I	I	I	I	I	I

Program berhasil mendeteksi bahwa input tidak valid karena panjang baris tidak konsisten, dan menampilkan pesan error pada panel status GUI.

Karena input tidak valid, program tidak menghasilkan file solusi .txt. Output hanya berupa pesan error yang ditampilkan di GUI.

Output sebagai Gambar (PNG)



Gambar 3.5.1: Output GUI — Test 5 (Input Tidak Valid)

4 Lampiran

https://github.com/Joji0/Tucil1_13524023

Tabel 4.1: Evaluasi

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil di jalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Program dapat menyimpan solusi dalam bentuk file gambar	✓	

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Jonathan Kris Wicaksono [13524023]